

Smart Mental Wellness Tracking & Analytics System

Complete 14-Day Project Roadmap · Version 2.0 — Enhanced Edition

Nethara Vidmanthi | 23CDS0847 | B.Sc. Data Science | SUSL

Python · Flask · MySQL · Flask-Bcrypt · Chart.js · ReportLab · pytest · GitHub

★ NOW INCLUDES

Admin Dashboard & Counselor View · Data Science Analytics Depth · Security (Bcrypt + HTTPS) · Full UAT Testing Plan

■ Project At-a-Glance

Student	Nethara Vidmanthi Reg No: 23CDS0847
Degree & Module	B.Sc. (Hons) Data Science · Capstone Project in Data Science I (DS2105)
University	Sabaragamuwa University of Sri Lanka — Department of Data Science, Faculty of Computing
Timeline	14 Days (2 Weeks) — structured daily task plan
Core Tech	Python 3, Flask, MySQL, SQLAlchemy, Jinja2 templates, Bootstrap 5
Security	Flask-Bcrypt (password hashing), Flask-Login (sessions), CSRF protection, .env secrets
Analytics	Chart.js (interactive), Matplotlib (static/PDF), Pandas (statistics), rule-based risk engine
Testing	pytest (unit + integration), Manual UAT with test users, Browser cross-testing
Deliverables	✓ Working Web App ✓ PDF Export ✓ Admin Panel ✓ Final Report ✓ Presentation Slides
Version Control	GitHub — daily commits, meaningful messages, clean README, v1.0 release tag

■ Project Objectives (Updated)

- Develop a **secure, role-based web application** — student self-tracking + admin/counselor oversight panel
- Collect and manage wellness data efficiently: mood, stress, sleep quality, sleep hours, academic workload, notes
- Perform **data science analytics** — statistical analysis, correlation, trend detection, wellness score calculation

④	Automatically identify at-risk students using a rule-based risk engine and alert counselors/admins
⑤	Implement data privacy & security — Bcrypt hashing, input validation, access control, sensitive data protection
⑥	Generate downloadable PDF wellness reports and interactive visual dashboards with Chart.js

■ Two-Week Overview				
Phase	Days	Focus Area	Key Deliverables	New in v2
PHASE 1 Foundation	Days 1–7	Setup → DB → Auth (Bcrypt) → Data Entry → Dashboard → Analytics	Repo, schema, secure login, wellness forms, Chart.js dashboard, risk alerts	Bcrypt security, 6 objectives, risk engine
PHASE 2 Delivery	Days 8–14	Admin Panel → PDF Export → UAT Testing → Final Report → Slides	Admin counselor view, PDF export, test suite, 7-chapter report, presentation	Admin dashboard, UAT plan, DS analytics depth

WEEK 1 — Foundation, Security & Core Features

DAY 1

Project Setup & Environment

GitHub + VS Code + Flask + Security Libraries

- ■ **GitHub repository** — create repo with README.md, add Python .gitignore (includes venv/, __pycache__/, .env, instance/), initial commit "Day 1: Project init"
- **Verify VS Code folder structure:** templates/, static/css/, static/js/, static/charts/, venv/ — create any missing folders, add empty .gitkeep files
- **Activate virtual environment** — run: `python -m venv venv` then activate. Install all dependencies:
- **pip install:** `flask flask-login flask-sqlalchemy flask-bcrypt pymysql python-dotenv reportlab matplotlib pandas pytest` — then `pip freeze > requirements.txt`
- ■ **Project file structure:** app.py (factory), config.py (loads .env), models.py, .env (SECRET_KEY, DB_URI — never commit this)
- **.env file setup:** SECRET_KEY=your-random-string-here, DATABASE_URL=mysql+pymysql://user:pass@localhost/mental_wellness_db, FLASK_ENV=development
- **Flask hello world:** verify app runs on localhost:5000 — "/" returns "System Online"
- **Git commit:** "Day 1: Project scaffold, Flask setup, all dependencies installed"

DAY 2

Database Design & MySQL Setup

ER Diagram → Schema → SQLAlchemy Models

- ■ **MySQL setup:** start MySQL service → `CREATE DATABASE mental_wellness_db CHARACTER SET utf8mb4;` → create user and `GRANT ALL PRIVILEGES`
- **ER Diagram (draw.io / paper):** entities: users, wellness_logs, mood_categories — include all attributes, primary keys, foreign keys, relationships
- **users table:** id (PK), name, email (UNIQUE), password_hash (VARCHAR 255), reg_number (UNIQUE), role (ENUM: student/admin/counselor, default student), created_at, is_active (BOOL)
- **wellness_logs table:** id (PK), user_id (FK → users.id), log_date (DATE, NOT NULL), mood_score (TINYINT 1-5), stress_level (TINYINT 1-5), sleep_hours (DECIMAL 3,1), sleep_quality (TINYINT 1-5), academic_workload (TINYINT 1-5), notes (TEXT), wellness_score (DECIMAL 5,2 — computed), created_at
- **SQLAlchemy models.py:** User class (UserMixin, role field, relationship to logs), WellnessLog class (all fields, backref to user)
- **Create tables:** Flask shell → `from app import db; db.create_all()` → verify in MySQL CLI with `SHOW TABLES`
- **Git commit:** "Day 2: DB schema, ER diagram, SQLAlchemy models"

- **Flask-Bcrypt init:** `from flask_bcrypt import Bcrypt; bcrypt = Bcrypt(app)` — this replaces Werkzeug hashing and is more secure
- **Registration /register:** HTML form (name, reg_no, email, password, confirm_password), server-side: check email not duplicate, validate password min 8 chars + 1 number, hash with `bcrypt.generate_password_hash(password).decode("utf-8")`, set role=student by default
- **Login /login:** look up user by email, verify with `bcrypt.check_password_hash(user.password_hash, form_password)` — NEVER compare plaintext, call `login_user(user)`
- ■ **Role-based access decorators:** create `admin_required` and `counselor_required` decorators that check `current_user.role` — protect all `/admin/*` and `/counselor/*` routes
- **CSRF protection:** add WTForms or manual CSRF token to all POST forms: `from flask_wtf.csrf import CSRFProtect; csrf = CSRFProtect(app)`
- **Logout /logout:** `logout_user()`, clear session, redirect to `/login`
- **base.html:** Bootstrap 5 CDN, navbar showing username + role badge, logout link, flash message area for success/error/warning
- **Git commit:** "Day 3: Secure auth with Flask-Bcrypt, role-based access control"

■ **SECURITY NOTE:** *Flask-Bcrypt uses bcrypt algorithm with automatic salt generation — far stronger than plain SHA-256. Always decode the hash to string before storing: `bcrypt.generate_password_hash(pw).decode("utf-8")`. Mention this explicitly in your Final Report Chapter 4 (Implementation) to score well on Security criteria.*

DAY 4 Wellness Data Entry Module*Form → Validation → Bcrypt-Protected Save → CRUD*

- **Wellness entry form /log (GET):** HTML with range sliders for mood (1-5, emoji labels ■■■■■), stress (1-5, color coded green→red), sleep hours (3.0–12.0, step 0.5), sleep quality (1-5), academic workload (1-5), date picker (default today), optional notes textarea
- **Wellness Score auto-calculation (Python):** $\text{score} = (\text{mood_score} * 20 + (6 - \text{stress_level}) * 20 + \text{sleep_quality} * 20 + \min(\text{sleep_hours} / 8, 1) * 20 + (6 - \text{academic_workload}) * 20) / 5$ → range 0–100. Save this to `wellness_logs.wellness_score` column
- ✓■ **Server-side validation (NEVER trust client only):** all scores 1–5 range check, `sleep_hours` 0–16 range, `log_date` not in future, duplicate `log_date` per user check → show specific error flash messages
- **POST /log:** create `WellnessLog` object, compute `wellness_score`, `db.session.add()`, `db.session.commit()` → redirect to /dashboard with success flash
- **History page /history:** query `current_user` logs ordered by `log_date` DESC, paginate 10/page using Flask-SQLAlchemy `paginate()`, display as styled table with `wellness_score` color badge (green ≥70, amber 40–69, red <40)
- **Edit /log/edit/:** ownership check (`log.user_id == current_user.id`), pre-fill form, recalculate score on save
- **Delete /log/delete/:** ownership check, POST method only (GET delete is insecure), confirm modal in JS
- **Git commit:** "Day 4: Wellness CRUD, score formula, server-side validation"

DAY 5 Dashboard & Interactive Charts*Chart.js → Trends → Summary Cards → Risk Banner*

- **/dashboard route:** query last 30 days of `wellness_logs` for `current_user`, compute summary stats using Python (not just display raw data)
- **Mood trend line chart (Chart.js):** pass data as JSON: `json.dumps({"labels": [dates], "data": [moods]})`, render in Jinja2 template, Chart.js canvas element — smooth bezier curves, teal color
- **Stress bar chart (Chart.js):** weekly averages computed in Python using pandas or list comprehension, color each bar dynamically: green <3, orange 3-4, red >4
- **Sleep dual-axis chart (Chart.js):** `sleep_hours` as bar (left axis), `sleep_quality` as line (right axis) — shows correlation visually
- **Wellness Score trend line (Chart.js):** plot daily `wellness_score` over 30 days — this is your computed data science metric, highlight it prominently
- **Summary stat cards (4 cards):** Avg Mood This Week (with emoji), Avg Stress This Week (color coded), Consecutive Logging Streak (days), Overall Wellness Score (0–100 progress bar)
- **Risk alert banner:** Python logic: if `any(stress ≥ 4 for last 5 consecutive days)` → show red banner "■■ High stress detected for 5+ days. Consider speaking to a university counselor." — this is your Rule-Based Risk Engine
- **Git commit:** "Day 5: Dashboard, 4x Chart.js charts, risk alert banner"

DAY 6

Data Science Analytics Depth

Statistics → Correlation → Insights → Weekly Report

★ NEW

- **Statistical analysis (pandas):** from wellness_logs data compute: mean, median, std deviation for each metric — display as "Your Statistics" section on dashboard
- **Pearson correlation analysis:** `import scipy.stats; r, p = scipy.stats.pearsonr(sleep_hours_list, stress_list)` → display: "Sleep hours and stress level show $r=-0.72$ (strong negative correlation)" — this IS data science
- **Day-of-week pattern analysis:** group logs by weekday, compute avg stress per day → "Your highest stress day is Monday (avg 4.2/5)" — use pandas groupby
- **Rule-Based Risk Engine (document this in report!):** Rule 1: $\text{avg_stress} \geq 4.0$ last 7 days → "Chronic Stress Alert". Rule 2: $\text{sleep_hours} < 6.0$ for 3+ consecutive days → "Sleep Deprivation Alert". Rule 3: wellness_score dropping >20 points in a week → "Wellness Decline Alert". Rule 4: no log entry for 5 days → "Engagement Alert"
- **Weekly report page /report:** week selector (dropdown), table of daily metrics, week avg vs. previous week (% change with arrow), personalized text recommendations based on rules
- **Matplotlib static charts (for PDF):** generate mood_trend.png, stress_chart.png, sleep_chart.png into `static/charts/[user_id]/` folder — one set per user
- **Git commit:** "Day 6: DS analytics, Pearson correlation, rule-based risk engine"

■ **DATA SCIENCE MARKS:** *The correlation analysis and rule-based risk engine are what differentiate a Data Science project from a simple web app. In your Final Report Chapter 3 (Specification & Design), create a dedicated sub-section: "3.4 Data Analysis Engine" describing the algorithms. In Chapter 5 (Results), show actual correlation values from your test data. This directly targets the "Innovation & Originality" and "Methodology" rubric criteria.*

DAY 7

Week 1 Review & Polish

Full Walkthrough → Bug Fix → Supervisor Demo

- **Full walkthrough test:** register → login → add 7 days of test data (manually enter) → check dashboard charts populate → check risk banner triggers → view history → check edit/delete → logout
- **Responsive check:** Chrome DevTools → mobile view (iPhone SE 375px, Galaxy 360px) — ensure sliders, charts, cards all render on small screens
- **Bug fixing session:** fix any broken routes, 500 errors, missing `@login_required`, form validation gaps
- **Security audit:** confirm `.env` not in git, all admin routes protected, ownership checked on edit/delete, no SQL injection (using ORM = safe)
- **UI polish:** add favicon (simple brain/heart icon), proper tags on all pages, 404.html error page
- **Supervisor Week 1 update:** send email with GitHub link + 3 screenshots (login page, dashboard with charts, history table). Write: "Completed: auth, data entry, dashboard with 4 charts, risk alert. Starting admin panel + PDF export this week."
- **Git commit:** "Day 7: Week 1 complete — auth, entry, dashboard, risk engine all working"

WEEK 2 — Admin Panel, Testing, Report & Submission

DAY 8 PDF Report Export Feature

ReportLab → Downloadable Wellness Report

- **pip install reportlab** — already in requirements.txt from Day 1. Create /export/pdf route, accept ?week=YYYY-WW or ?month=YYYY-MM query param
- **PDF content:** Page 1: SUSL-style header (university name, dept, project title), student name + reg_no, report date range, 4 summary stat boxes. Page 2: embedded Matplotlib chart images from static/charts/[user_id]/. Page 3: daily metrics table (date, mood, stress, sleep, workload, wellness score)
- **Professional PDF styling:** navy header bar, teal accent tables, wellness score color-coded (green/amber/red), proper margins — matches your project color scheme
- **Flask response:** from flask import send_file; return send_file(pdf_buffer, as_attachment=True, download_name=f"wellness_{user.reg_number}_{date}.pdf")
- **Add export button** on /report page: "■ Download PDF Report" with date range form
- **Security:** only allow current_user to download their own report — check user_id parameter if any
- **Git commit:** "Day 8: PDF export with ReportLab, professional formatting"

DAY 9 Admin & Counselor Dashboard

Role-Based Views → At-Risk Detection → Aggregated Analytics

★ NEW

- **Create admin + counselor accounts:** Flask shell → admin = User(email="admin@susl.ac.lk", role="admin"); admin.password_hash = bcrypt.generate_password_hash("AdminPass1").decode("utf-8"); db.session.add(admin); db.session.commit()
- **/admin/dashboard:** system overview cards — Total Students Registered, Total Logs This Week, Students At-Risk (stress ≥ 4 for 5 days), Average System Wellness Score today
- **/admin/students:** table of ALL students — columns: Name, Reg No, Total Logs, Last Log Date, 7-Day Avg Stress, Wellness Score, Risk Status. Sort by risk (worst first). Rows color-coded: red = at-risk, amber = watch, green = healthy
- **/admin/student/:** view ANY student's full dashboard (same charts, history, analytics) — for counselor to review and act. Add "Flag for Counseling" button
- **Aggregated analytics /admin/analytics:** system-wide charts — avg wellness score over time (all students combined), stress distribution histogram, most common risk alerts triggered, day-of-week stress heatmap across all students
- **At-risk alert list /admin/alerts:** list all students currently flagged by risk engine, show which rule triggered (Chronic Stress / Sleep Deprivation / Wellness Decline / No Engagement), show since when
- **All /admin/* routes:** decorated with admin_required — non-admins get 403 Forbidden page
- **Git commit:** "Day 9: Admin dashboard, counselor view, at-risk student tracking"

■ **WHY THIS MATTERS:** *The Admin/Counselor Dashboard transforms your project from a "student diary app" into a "complete organizational mental health management system." This directly addresses the evaluator feedback about adding institutional value. Highlight this in your presentation: "This system gives university counselors real-time visibility into student wellness trends without accessing individual sensitive entries — privacy by design."*

- **Unit tests (pytest) — test_unit.py:** test_wellness_score_calculation() → assert score == expected for known inputs. test_risk_engine_chronic_stress() → feed 5 days of stress=5, assert risk flag triggered. test_bcrypt_hash() → hash a password, verify check_password_hash returns True. test_invalid_score_validation() → score=6 should fail validation
- **Integration tests — test_integration.py:** use Flask test client (app.test_client()): test_register_and_login() → POST to /register then /login, assert redirect to dashboard. test_log_entry_saves_to_db() → login as test user, POST /log, query DB, assert record exists. test_admin_cannot_see_other_user_logs() → login as student, GET /admin/students, assert 403
- **UAT Test Plan — test_uat.md (document in repo):** recruit 2-3 classmates as testers. Provide UAT script with numbered steps they follow. Collect their feedback on a Google Form
- **UAT Script (give to testers):** 1. Register with your university email. 2. Log today's wellness (all fields). 3. Check your dashboard — do the charts make sense? 4. Download a PDF report. 5. Rate ease of use 1-5. 6. What would you improve?
- **Document UAT results** in a table: Tester ID, Task, Pass/Fail, Comments — this goes in Final Report Chapter 5 (Results & Evaluation)
- **Performance testing:** time dashboard load with 30 days of data — target < 2 seconds. If slow, add DB index: `db.Index("idx_user_date", WellnessLog.user_id, WellnessLog.log_date)`
- **Bug log:** maintain bugs.md with: Bug ID, Description, Severity (Critical/High/Medium/Low), Status (Fixed/Open). Fix all Critical and High before submission
- **Git commit:** "Day 10: Unit tests, integration tests, UAT conducted and documented"

■ **UAT FOR YOUR REPORT:** In Chapter 5 (Results & Evaluation), create a table showing your UAT results. Include: number of testers, tasks tested, pass rate, feedback themes, and how you addressed issues. This demonstrates professional evaluation practice and targets the "Demonstration of software system working" rubric criterion (20 marks in the final presentation rubric).

- **Chapter 1: Introduction** — project overview paragraph, motivation (student mental health crisis with statistics from literature), 6 SMART objectives, scope (web-based, SUSL students, no ML — future work), approach summary, concise major outcomes. Length: 2-3 pages
- **Chapter 2: Background** — review 4-5 existing systems: MoodTracker app, Wysa, DASS-21 (questionnaire), university wellness portals. For each: what it does, what it lacks. Conclude with gap analysis: "None provide real-time admin oversight + integrated data science analytics for a university context — this project fills that gap." Min 4 IEEE references
- **Chapter 3: Specification & Design** — 3.1 Functional Requirements (numbered list FR1-FR12), 3.2 Non-Functional Requirements (security, performance, usability, reliability), 3.3 Use Case Diagram (student actor + admin/counselor actor — draw with draw.io, export PNG), 3.4 Data Analysis Engine (describe wellness score formula + 4 risk rules formally), 3.5 ER Diagram (from Day 2), 3.6 System Architecture (3-tier: browser → Flask → MySQL)
- **STRICT formatting:** Times New Roman 12pt, A4, 35mm left, 30mm right/top/bottom, 1.15 line spacing, all figures captioned "Figure N: Description", all referenced in text, IEEE references format
- **Git commit:** "Day 11: Final report Ch 1-3 draft" + save Final_Report_v1.docx to /docs/

- **Chapter 4: Implementation** — 4.1 Technology Selection (Flask vs Django justification, MySQL vs SQLite, Flask-Bcrypt for security), 4.2 Security Implementation (bcrypt details, role-based access, CSRF, input validation — this is new content!), 4.3 Data Analysis Implementation (wellness score code snippet, correlation code, risk engine pseudocode), 4.4 Key Challenges & Solutions (list 3-4 real challenges you faced)
- **Chapter 5: Results & Evaluation** — 5.1 Feature Demonstration (screenshots of every screen with captions), 5.2 Wellness Score Validation (test with known inputs, verify formula correctness), 5.3 Correlation Analysis Results (show actual r values from your test data), 5.4 UAT Results (table from Day 10), 5.5 Risk Engine Evaluation (show rule triggers with test scenarios), 5.6 Limitations (no real-time notifications, no ML, no mobile app yet)
- **Chapter 6: Future Work** — ML-based mood prediction (scikit-learn), mobile app (Flutter/React Native), SMS/email alert system for counselors, integration with SUSL student management system, multi-language support (Sinhala/Tamil/English)
- **Chapter 7: Conclusions** — restate each of the 6 objectives and whether ACHIEVED or PARTIALLY ACHIEVED, significance of the security and data science depth, reflection on what you learned, final statement on project impact
- **References:** IEEE format — min 8 references total: mix of academic papers (Google Scholar), Flask/Python docs, existing system papers
- **Appendices:** Appendix A — Declaration, Appendix B — Certificate of Approval, Appendix C — UAT feedback forms, Appendix D — selected code listings
- **Git commit:** "Day 12: Final report Ch 4-7 complete, references, appendices"

DAY 13**Presentation Slides & App Final Polish***Slides → Demo Script → UI Cleanup*

- ■ **Presentation structure (12-15 slides):** 1. Title Slide (name, reg, supervisor), 2. Problem Statement (stats on student mental health), 3. Objectives (6 objectives on one clean slide), 4. System Architecture Diagram, 5. Technology Stack & Justification, 6. Security Implementation (Bcrypt highlight), 7. Live Demo — Student Flow, 8. Dashboard & Charts (show all 4), 9. Data Science Analytics (correlation + risk engine), 10. Admin/Counselor Dashboard (new!), 11. Testing Results (UAT table), 12. Results vs Objectives, 13. Future Work, 14. Conclusions, 15. Q&A;
- **Slide design:** professional SUSL-branded theme, navy + teal colors matching your app, university logo on title, consistent fonts (Calibri or Arial), avoid walls of text — use icons and diagrams
- **Demo script (prepare step-by-step):** 1. Show login page (mention Bcrypt security), 2. Register new student (show validation), 3. Add 7 sample wellness logs quickly (have test account pre-loaded), 4. Show dashboard with all 4 charts, 5. Trigger risk alert (show high-stress scenario), 6. Show weekly report + PDF download, 7. Login as admin, 8. Show student overview table (at-risk highlighted), 9. Show aggregated analytics, 10. Q&A; ready
- **Final app polish:** remove all debug print() statements, set FLASK_DEBUG=False in production config, remove test data from main DB (keep test DB separate), ensure all flash messages appear correctly
- **Final responsive check:** test on Chrome (desktop + mobile), Firefox — fix any display issues
- **Submission package prep:** Final_Report.pdf, Source_Code/ (clean copy of project), Presentation.pptx, README.md with full setup steps, requirements.txt
- **Git commit:** "Day 13: Presentation slides, final app cleanup, submission package ready"

DAY 14**Final Review & Submission***Last Checks → Submit → Practice Presentation*

- **Final feature checklist:** ✓ Register/Login (Bcrypt) ✓ Wellness log entry (all 5 metrics) ✓ Dashboard (4 Chart.js charts) ✓ Summary stat cards ✓ Risk alert banner ✓ Wellness score trend ✓ Correlation analysis displayed ✓ Weekly report ✓ PDF export ✓ Admin student list ✓ Admin at-risk highlight ✓ Counselor view per student ✓ History + edit + delete ✓ All routes @login_required ✓ Role-based access working
- **Final report review:** proofread all 7 chapters + appendices, verify ALL figures have captions and are referenced in text, check page numbers (Roman for front matter, Arabic from Ch.1), count references (min 8), run spell-check
- ■ **Approval documents:** print Declaration (Appendix A) and Certificate of Approval (Appendix B), get supervisor signature on Certificate before submission
- **Final submission package:** zip source code (exclude venv/ and .env), convert report to PDF, organize: /submission/Final_Report.pdf, /submission/Source_Code.zip, /submission/Presentation.pptx, /submission/README.txt
- ■ **Presentation rehearsal:** 2 full practice runs — time yourself (target 10-12 minutes), practice the live demo flow, prepare answers for likely Q&A: "Why Flask?", "How does the risk engine work?", "How did you test?", "What would you add next?", "How is data kept private?"
- **Supervisor sign-off:** demo full product to supervisor before submitting — get verbal/written approval
- **Final GitHub commit + release:** git tag v1.0 && git push origin v1.0 — "v1.0 Final Submission — Smart Mental Wellness Tracking & Analytics System"
- **Submit to portal** → confirm submission receipt → you are done! ■

■ Supervisor Rubric Alignment — Proposal & Final Evaluation

Proposal Evaluation Rubric (30% of total marks)

Criterion	Marks	What You Are Doing	Target Days
a. Problem Statement & Motivation	10	Dashboard risk alerts + at-risk student system prove real-world impact. Ch.1 motivation uses literature stats	Day 11
b. Objectives & Scope	10	6 SMART objectives defined (expanded from 4), clear scope boundary (SUSL web-based, no ML in scope)	Days 1, 11
c. Innovation & Originality	10	Wellness Score formula, Pearson correlation, 4-rule risk engine, admin counselor view — all novel for SUSL context	Days 6, 9
d. Technical Feasibility	10	Full tech stack installed Day 1, working app by Day 7 proves feasibility. Bcrypt security adds professional depth	Days 1–7
e. Methodology & Approach	15	ER diagram, Use Case diagram, risk engine pseudocode, 14-day structured plan, UAT, pytest — all documented	Days 2,3,10,11
f. Expected Outcomes & Impact	20	6 outcomes: student wellness tracking, admin oversight, risk identification, PDF reports, analytics, security	Days 5,6,8,9
g. Timeline & Milestones	10	This 14-day roadmap IS your Gantt chart — present it. Daily milestones, supervisor check-ins documented	This document
h. Documentation Quality	15	Report follows SUSL guidelines exactly: Times New Roman 12, margins, IEEE refs, all figures captioned	Days 11,12,13

Final Presentation Evaluation Rubric (70% of total marks)

Criterion	Marks	How to Score Full Marks
a. Design Requirements & Implementation	10	Explain system architecture clearly, justify every tech choice (Bcrypt not Werkzeug, Flask not Django, MySQL not SQLite)
b. Technology Evaluation & Selection	10	In slides: show alternatives considered, explain WHY you chose each. e.g. "Bcrypt vs MD5 — bcrypt is adaptive, more secure"
c. Creativity & Innovation	15	Demo the risk engine live, show correlation results, highlight admin counselor view — these are your innovation differentiators
d. Software Working & Functioning	20	All 6 objectives demonstrated live. All modules integrated (student ↔ admin ↔ analytics ↔ PDF). Zero crashes during demo
e. Quality of Work Done	20	Show you understand EVERYTHING — explain wellness formula, risk rules, correlation, Bcrypt. Do not read slides — know your code
f. Professionalism & Presentation Quality	15	Wear formal attire, maintain eye contact, manage 12 minutes, clear voice. Practice 3× before the day
g. Handling Q&A;	10	Prepare answers for: risk engine logic, security choices, testing approach, what you would change, privacy of mental health data

ADDITIONAL RECOMMENDATIONS

Implementation Guide

Mental health data is sensitive — handle it with professional care

- ■ **Flask-Bcrypt (Day 3):** pip install flask-bcrypt. Hash on register: `bcrypt.generate_password_hash(pw).decode("utf-8")`. Verify on login: `bcrypt.check_password_hash(stored_hash, pw)`. NEVER store plaintext or MD5/SHA1 hashes
- ■ **CSRF Protection (Day 3):** pip install flask-wtf. Add `CSRFProtect(app)`. Add `{{ csrf_token() }}` hidden field in every form. Without this, your forms are vulnerable to cross-site request forgery
- ■ **Input Validation (Day 4):** Validate ALL inputs server-side — NEVER trust JavaScript validation alone. Check score ranges, date ranges, string lengths. Use parameterized queries (SQLAlchemy ORM handles this automatically)
- ■ **Session Security (Day 3):** Set `SESSION_COOKIE_SECURE=True` (HTTPS), `SESSION_COOKIE_HTTPONLY=True` (prevent JS access), `PERMANENT_SESSION_LIFETIME = timedelta(hours=2)` for auto-logout
- ■ **Access Control (Day 3,9):** Every route accessing user data MUST check `current_user.id` matches the data owner. Admin routes MUST check `current_user.role == "admin"`. Never pass `user_id` as URL param without ownership verification
- ■ **Privacy by Design (Admin Panel - Day 9):** Admin sees aggregated data and at-risk flags — NOT individual diary entries. Individual notes field is only visible to the student who wrote it. Mention this in your report as "Privacy by Design" principle
- ■ **Report Mention (Day 11-12):** Dedicate a sub-section in Chapter 4: "4.2 Security Implementation" — describe bcrypt, CSRF, access control, session management. This can earn significant marks under the Technical Feasibility and Methodology rubric criteria

Code Snippets to Implement

Turn your web app into a Data Science project — these are must-haves

Wellness Score Formula (Day 4)

```
score = round((mood_score*20 + (6-stress_level)*20 + sleep_quality*20 +  
min(sleep_hours/8,1.0)*20 + (6-academic_workload)*20) / 5, 2) # Returns 0-100
```

Pearson Correlation (Day 6)

```
from scipy.stats import pearsonr  
sleep = [log.sleep_hours for log in logs]  
stress = [log.stress_level for log in logs]  
r, p_value = pearsonr(sleep, stress)  
# r < -0.5 means "sleep and stress are negatively correlated" - show this insight to user
```

Weekly Pandas Aggregation (Day 6)

```
import pandas as pd  
df = pd.DataFrame([l.log_date, l.stress_level, l.mood_score) for l in logs],  
columns=["date","stress","mood"])  
df["weekday"] = pd.to_datetime(df["date"]).dt.day_name()  
print(df.groupby("weekday")["stress"].mean()) # Finds most stressful day
```

Risk Engine - Rule 1 Chronic Stress (Day 6)

```
recent =
WellnessLog.query.filter_by(user_id=uid).order_by(WellnessLog.log_date.desc()).limit(7).all()
if len(recent)>=5 and all(log.stress_level>=4 for log in recent[:5]):
alerts.append({"type":"CHRONIC_STRESS","message":"Stress ≥4 for 5+ days"})
```

Risk Engine - Rule 2 Sleep Deprivation (Day 6)

```
if sum(1 for log in recent[:3] if log.sleep_hours < 6.0) == 3:
alerts.append({"type":"SLEEP_DEPRIVATION","message":"Sleep < 6h for 3 consecutive days"})
```

Risk Engine - Rule 3 Wellness Decline (Day 6)

```
if len(recent)>=7:
this_week = sum(l.wellness_score for l in recent[:7])/7
last_week_logs = WellnessLog.query.filter_by(user_id=uid).order_by(WellnessLog.log_date.desc()).
offset(7).limit(7).all()
if last_week_logs:
last_week = sum(l.wellness_score for l in last_week_logs)/len(last_week_logs)
if last_week - this_week > 20:
alerts.append({"type":"WELLNESS_DECLINE"})
```

Daily commits: At minimum one commit per day. Format: "Day [N]: [Brief description of what was completed]" —

→ evaluators see this timeline as proof of progress

Meaningful .gitignore: Ensure these are NEVER committed: venv/, __pycache__/, .env, instance/, *.pyc, static/charts/

→ (generated files), *.db. Add them on Day 1 before any other commits

README.md quality: Include: Project title + screenshot, Prerequisites (Python 3.x, MySQL), Setup steps (clone → venv → pip install → .env setup → flask db → python app.py), Test accounts (admin/student credentials for evaluator),

→ Feature list with screenshots

Branch discipline: For this project scale: work on main is acceptable. But create a test branch on Day 10 for pytest —

→ keep main always in working state

v1.0 Release tag: Day 14: git tag -a v1.0 -m "Final Submission" && git push origin v1.0. Create a GitHub Release with

→ release notes listing all features — looks very professional

Progress log file: Create docs/progress_log.md: | Date | Completed | Next | Blockers | Show this to supervisor at each

→ check-in

■ ■ Risk Register & Mitigation Plan

Risk	Likelihood	Impact	Mitigation
MySQL connection error on startup	High	High	Day 2: test connection immediately. Keep MySQL as Windows service or use: mysql.server start. Store connection string in .env
Bcrypt import error	Low	High	Ensure flask-bcrypt in requirements.txt (not just bcrypt). Import: from flask_bcrypt import Bcrypt
CSRF token missing in forms	Medium	Medium	Add {{ form.hidden_tag() }} or manual csrf_token() to EVERY POST form. Test all forms on Day 7
Admin sees student private notes	Medium	Very High	In admin queries: only select aggregated columns, never show notes field in admin views. Test on Day 9
Report left to Day 13-14	Very High	Very High	CRITICAL: Start Chapter 1 on Day 11 exactly as planned. Take screenshots daily as you build
Chart.js data not populating	Medium	Medium	Always pass Python list as JSON: json.dumps(list). Check browser console (F12) for JS errors on Day 5
scope creep — adding too many features	High	High	Finish ALL 6 core objectives before any enhancements. The roadmap order is not optional
Virtual environment broken	Medium	Medium	If venv corrupted: rm -rf venv → python -m venv venv → pip install -r requirements.txt. 10 min fix
Scipy/Pandas not installed	Medium	Medium	Add to requirements.txt Day 1: scipy pandas. Test import on Day 6 before building analytics
UAT testers unavailable	Medium	Medium	Ask classmates on Day 9 — give them 1-2 days notice. Even 2 testers + their feedback is sufficient for report

Enhancement	Innovation Value	Difficulty	Add on Day
CSV Data Export	Lets users download their raw data — real data science tool	Easy	Day 8 buffer
Email Alerts to Counselors	When risk engine fires, auto-email counselor — closes the loop	Medium	Day 9 buffer
Mood Heatmap Calendar	GitHub-style heatmap of daily wellness scores — visual wow factor	Medium	Day 6 ext.
Simple Linear Regression	Predict next week stress from this week data — real ML basics	Medium	Day 6 ext.
Data Anonymization	Admin exports anonymized dataset — data ethics demonstration	Easy	Day 9 ext.
Flask-Mail Integration	Weekly wellness summary email to students — professional feature	Medium	Day 9 buffer

■ Complete Final Submission Checklist

- **Working Web Application** — All 6 objectives functional, no crashes during demo, all routes secure
- **Flask-Bcrypt Security** — Password hashing implemented, CSRF protection on all forms, role-based access enforced
- **Admin/Counselor Dashboard** — Student overview, at-risk highlighting, aggregated analytics, per-student view
- **Data Science Analytics** — Wellness score, Pearson correlation, risk engine (4 rules), day-of-week analysis displayed
- **PDF Export Feature** — Downloadable report with charts, student details, metric tables
- **MySQL Database** — All tables created, sample data loaded, indexes on user_id + date
- **pytest Test Suite** — Unit tests + integration tests passing, UAT documented in test_uat.md
- **Source Code on GitHub** — Daily commits visible, meaningful messages, README with setup, v1.0 release tag
- **Final Report — 7 Chapters** — Times New Roman 12, 35mm left margin, IEEE refs (min 8), all figures captioned
- **Security Sub-section in Report** — Ch 4.2 explicitly covers: Bcrypt, CSRF, access control, session security, privacy by design
- **UAT Results Table in Report** — Ch 5.4 shows tester feedback, pass/fail, improvements made
- **Presentation Slides (12-15)** — All rubric criteria covered, demo script rehearsed, 12 min timing practiced 3×
- **Declaration Page (Appendix A)** — Signed by you
- **Certificate of Approval (Appendix B)** — Signed by supervisor and HOD
- **Progress Log submitted** — docs/progress_log.md shows weekly supervisor check-ins

■ **Nethara, you have got a truly complete project blueprint now.** Every evaluator concern is addressed: security (Bcrypt + CSRF + role-based access), data science depth (correlation + risk engine + wellness score), admin/counselor dashboard, and a full UAT testing plan. Follow this roadmap day-by-day, commit to GitHub daily, and you will walk into your presentation with a product you genuinely understand and can confidently defend. The best projects are not the most complex — they are the most complete and clearly explained. Good luck! ■■